

Implementation of Convolutional Neural Network on FPGA for MNIST Digit Recognition

Engineering Thesis

Author Name

January 2026

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Scope of the Thesis	1
2	Training	3
2.1	LeNet-5 Historical Context	3
2.2	CNN Model Architecture	3
2.2.1	Layer Details	4
2.3	Dataset and Preprocessing	4
2.3.1	MNIST Dataset	4
2.3.2	Preprocessing Pipeline	5
2.4	Training Configuration	5
2.5	Quantization Strategy	5
2.5.1	Weight Quantization	6
2.5.2	Bias Quantization	6
2.5.3	Arithmetic Pipeline	6
3	Inference Implementation	7
3.1	Target Hardware	7
3.2	System Architecture	7
3.3	Inference FSM	8
3.3.1	Convolution Implementation	8
3.3.2	Max Pooling Implementation	9
3.4	Memory Architecture	9
3.4.1	Distributed RAM	9
3.4.2	Memory Organization	10
3.5	Communication Protocol	10
3.5.1	UART Router	10
3.5.2	UART Configuration	11
3.5.3	Protocol Markers	11
3.5.4	Binary Safety	11
3.5.5	Flow Control	11
4	Verification	12
4.1	Verification Strategy	12
4.2	Python Bit-Exact Simulation	12
4.3	Test Infrastructure	13
4.3.1	Comparison Framework	13
4.4	Verilog Testbench	13

4.5	Debug Infrastructure	13
4.5.1	Status LEDs	13
4.5.2	7-Segment Display	14
5	Results and Comparison	15
5.1	Classification Accuracy	15
5.1.1	Primary Results	15
5.1.2	Bit-Exact Verification	15
5.2	Resource Utilization	16
5.3	Performance Analysis	16
5.3.1	Timing	16
5.3.2	Power Consumption	16
5.4	Comparison with Simpler Models	17
5.5	Comparison with Related Work	17
5.6	Summary	17

Abstract

In the early days of artificial intelligence, neural networks were primarily trained on CPUs, which lacked the necessary power for deep architectures. The 2012 debut of AlexNet marked a turning point by leveraging the massive parallelism of GPUs to significantly accelerate matrix operations. However, the high energy demands of GPUs have driven a shift toward more efficient hardware, such as FPGAs and ASICs, to optimize inference for embedded systems.

This thesis presents a complete end-to-end implementation of a Convolutional Neural Network (CNN) for MNIST handwritten digit classification on a Basys3 FPGA board featuring the Xilinx Artix-7 XC7A35T chip. The project encompasses three main components: (1) training a two-layer CNN model using PyTorch with post-training static quantization to 8-bit integers, (2) implementing the inference engine in Verilog RTL as a 19-state finite state machine, and (3) comprehensive verification through Python simulation and hardware testing via UART communication.

Key contributions include a complete quantization pipeline that preserves accuracy while enabling efficient fixed-point arithmetic, a modular RTL architecture using distributed RAM for single-cycle memory access, and a robust UART protocol for weight loading and real-time inference. The implementation demonstrates the viability of deploying neural networks on resource-constrained FPGAs for embedded applications.

The system achieves 96.6% classification accuracy after quantization and maintaining bit-exact correspondence between Python simulation and FPGA hardware inference.

Keywords: FPGA, CNN, MNIST, Quantization, Verilog, Neural Network Acceleration, Embedded Systems

Chapter 1

Introduction

1.1 Background and Motivation

Deep neural networks demand extraordinary computational resources, performing billions of multiply-accumulate operations for a single inference. Traditional CPU architectures proved inadequate for such workloads, prompting the exploration of specialized accelerators. The watershed moment arrived in 2012 when AlexNet demonstrated that Graphics Processing Units could dramatically accelerate deep learning training through massively parallel processing, achieving breakthrough performance on the ImageNet classification challenge. This discovery catalyzed widespread adoption of GPUs for neural network development, fundamentally reshaping the AI research landscape and enabling the deep learning revolution.

Despite their training dominance, GPUs face significant challenges in deployment scenarios. Power consumption exceeding 200 watts, substantial physical size, and high cost make GPUs impractical for embedded and edge applications. Field-Programmable Gate Arrays have emerged as compelling alternatives for neural network inference, offering reconfigurable hardware that bridges the gap between software flexibility and hardware efficiency. FPGAs provide distinct advantages: energy consumption in single-digit watts, deterministic latency for real-time systems, support for custom bit-width arithmetic enabling aggressive quantization, and the ability to implement application-specific datapaths. These characteristics make FPGAs particularly suitable for resource-constrained deployments in autonomous systems, industrial automation, telecommunications infrastructure, and aerospace applications where power efficiency and reliability are paramount.

1.2 Scope of the Thesis

This work demonstrates a complete CNN implementation for MNIST digit classification on the Xilinx Artix-7 XC7A35T FPGA (Basys3 board). The system integrates three components: PyTorch-based training with post-training quantization to 8-bit integers, Verilog RTL inference engine implemented as a 19-state finite state machine, and comprehensive verification ensuring bit-exact correspondence between software simulation and hardware execution. The two-layer convolutional architecture processes 28×28 grayscale images through convolution, max pooling, and dense classification layers, with all weights and activations quantized to enable efficient fixed-point arithmetic on FPGA fabric.

Key contributions include a complete quantization workflow preserving classification accuracy while enabling hardware-friendly integer arithmetic, a modular RTL design uti-

lizing distributed RAM for single-cycle memory access, and a robust UART-based protocol for weight loading and real-time inference control. The implementation achieves 96.6% accuracy on MNIST classification, matches Python simulation bit-exactly across all test cases, completes inference in approximately 7.5 milliseconds (133 images per second), and consumes an estimated 0.4 watts of power. These results validate FPGA-based neural network deployment as viable for embedded applications where power efficiency and deterministic latency are critical requirements.

Chapter 2

Training

This chapter describes the training pipeline for the CNN model, including the neural network architecture, dataset preparation, training process, and the critical quantization strategy that enables efficient FPGA implementation.

2.1 LeNet-5 Historical Context

The CNN architecture implemented in this thesis draws inspiration from LeNet-5, the pioneering convolutional neural network developed by Yann LeCun and colleagues in the 1990s for handwritten digit recognition. LeNet-5 established several fundamental principles that remain relevant today:

- **Local Receptive Fields:** Small convolutional kernels that detect local features
- **Weight Sharing:** The same kernel applied across the entire input, reducing parameters
- **Subsampling (Pooling):** Spatial dimension reduction for translation invariance
- **Hierarchical Feature Extraction:** Progressive abstraction from edges to digits

While the original LeNet-5 used average pooling and tanh activations, modern implementations typically employ max pooling and ReLU activations for improved performance. This thesis implements a simplified two-layer CNN that captures these core principles while remaining tractable for FPGA implementation on resource-constrained devices.

2.2 CNN Model Architecture

The implemented model is a two-layer CNN optimized for FPGA deployment:

Table 2.1: CNN Layer Specifications

Layer	Input Shape	Output Shape	Parameters	Operations
Input	–	$1 \times 28 \times 28$	0	–
Conv1	$1 \times 28 \times 28$	$16 \times 26 \times 26$	160	97,344 MACs
MaxPool1	$16 \times 26 \times 26$	$16 \times 13 \times 13$	0	–
Conv2	$16 \times 13 \times 13$	$32 \times 11 \times 11$	4,640	557,568 MACs
MaxPool2	$32 \times 11 \times 11$	$32 \times 5 \times 5$	0	–
Flatten	$32 \times 5 \times 5$	800	0	–
Dense	800	10	8,010	8,000 MACs
Total			12,810	662,912 MACs

2.2.1 Layer Details

Convolutional Layer 1:

- 16 filters with 3×3 kernels
- No padding, stride 1
- ReLU activation
- Captures primitive features: edges, corners, curves

Convolutional Layer 2:

- 32 filters with 3×3 kernels across 16 input channels
- No padding, stride 1
- ReLU activation
- Combines primitive features into digit-specific patterns

Dense Layer:

- 800 input features from flattened pool output
- 10 output neurons (one per digit class)
- No activation (raw logits for classification)

2.3 Dataset and Preprocessing

2.3.1 MNIST Dataset

The MNIST database of handwritten digits contains:

- 60,000 training images
- 10,000 test images

- Each image: 28×28 pixels, grayscale (0-255)
- 10 classes (digits 0-9)

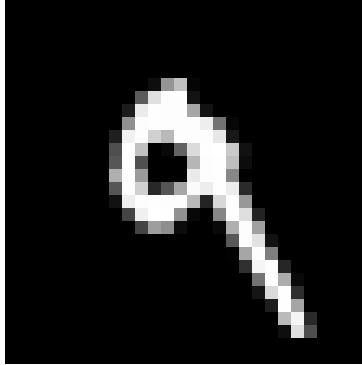


Figure 2.1: Example MNIST handwritten digit

2.3.2 Preprocessing Pipeline

The preprocessing transforms raw pixel values for neural network input:

$$x_{normalized} = \frac{x_{raw}/255 - \mu}{\sigma} \quad (2.1)$$

where $\mu = 0.1307$ and $\sigma = 0.3081$ are the MNIST dataset statistics.

For FPGA deployment, the normalized values are quantized to 8-bit signed integers:

$$x_{int8} = \text{clip}(\text{round}(x_{normalized} \times 127), -128, 127) \quad (2.2)$$

2.4 Training Configuration

The model is trained using the following hyperparameters:

Table 2.2: Training Hyperparameters

Parameter	Value
Batch Size	64
Epochs	5
Learning Rate	0.001
Optimizer	Adam
Loss Function	Cross-Entropy
Framework	PyTorch

2.5 Quantization Strategy

The project employs **post-training static quantization** to convert floating-point weights to integers while preserving accuracy. Post-training quantization (PTQ) was chosen over quantization-aware training (QAT) due to its simplicity in implementation, requiring no

retraining or modification to the training pipeline. For this MNIST application, PTQ achieved little to no accuracy drop, making the additional complexity of QAT unnecessary.

2.5.1 Weight Quantization

Weights are quantized to 8-bit signed integers:

$$W_{scale} = \frac{127.0}{\max(|W_{float}|)} \quad (2.3)$$

$$W_{int8} = \text{clip}(\text{round}(W_{float} \times W_{scale}), -128, 127) \quad (2.4)$$

2.5.2 Bias Quantization

Biases are quantized to 32-bit signed integers with scale propagation through layers:

Layer 1:

$$\text{InputScale} = 127.0 \quad (2.5)$$

$$\text{BiasScale}_{L1} = \text{InputScale} \times W_{scale,L1} \quad (2.6)$$

$$\text{Bias}_{L1,int32} = \text{round}(\text{Bias}_{L1,float} \times \text{BiasScale}_{L1}) \quad (2.7)$$

Layer 2 and Dense:

$$\text{OutputScale}_{L1} = \frac{\text{InputScale} \times W_{scale,L1}}{2^{SHIFT}} \quad (2.8)$$

$$\text{BiasScale}_{L2} = \text{OutputScale}_{L1} \times W_{scale,L2} \quad (2.9)$$

where $SHIFT = 8$ (division by 256).

2.5.3 Arithmetic Pipeline

The FPGA computes each convolution output as:

$$acc = bias + \sum_i (pixel_i \times weight_i) \quad (2.10)$$

Followed by the scaling and activation pipeline:

1. **Arithmetic Right Shift:** $temp = acc \gg 8$ (divide by 256)
2. **ReLU:** $if(temp < 0) \rightarrow temp = 0$
3. **Saturation:** $if(temp > 127) \rightarrow temp = 127$
4. **Store:** $output = temp[7 : 0]$

Chapter 3

Inference Implementation

This chapter describes the FPGA implementation of the CNN inference engine, including the hardware architecture, finite state machine design, memory organization, and communication protocols.

3.1 Target Hardware

The implementation targets the Digilent Basys3 development board, featuring:

Table 3.1: Target Hardware Specifications

Component	Specification
FPGA Board	Digilent Basys3
FPGA Chip	Xilinx Artix-7 XC7A35T-1CPG236C
Logic Cells	33,280
Block RAM	1,800 Kbit (225 KB)
DSP Slices	90
System Clock	100 MHz
UART Baud Rate	115,200 bps
Display	4-digit 7-segment
Debug LEDs	16 status LEDs

The Artix-7 family represents a cost-effective, low-power FPGA option suitable for educational and embedded applications, making it an appropriate platform for demonstrating neural network inference in resource-constrained environments.

3.2 System Architecture

The top-level design follows a modular architecture:

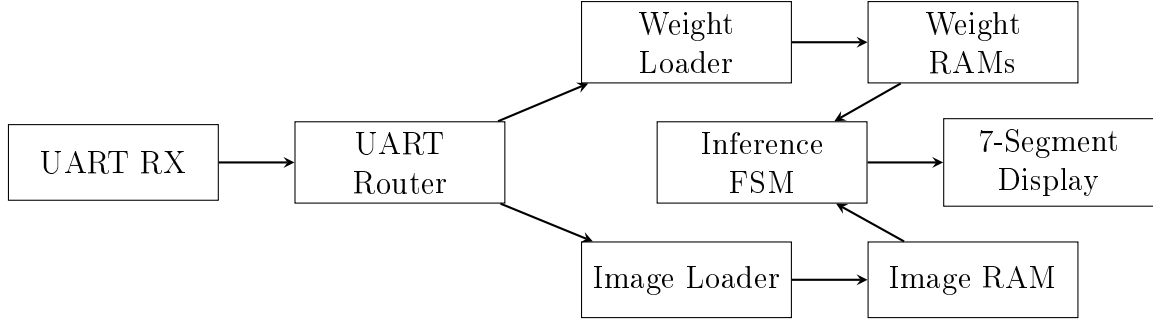


Figure 3.1: High-Level System Block Diagram

3.3 Inference FSM

The inference engine is implemented as a 19-state Finite State Machine:

Table 3.2: Inference FSM States

State	Name	Description
0	IDLE	Wait for start signal
1-2	L1_LOAD_BIAS*	Load Conv1 bias from BRAM
3	L1_PREFETCH	Prefetch first pixel/weight
4	L1_CONV	3×3 MAC operations
5	L1_SAVE	Apply shift/ReLU/saturation
6	L1_POOL	2×2 max pooling
7-8	L2_LOAD_BIAS*	Load Conv2 bias
9	L2_PREFETCH	Prefetch activation/weight
10	L2_CONV	3×3×16 MAC operations
11	L2_SAVE	Apply shift/ReLU/saturation
12	L2_POOL	2×2 max pooling
13-14	DENSE_LOAD_BIAS*	Load FC bias
15	DENSE_PREFETCH	Prefetch feature/weight
16	DENSE_MULT	800 MAC operations
17	DENSE_NEXT	Store score, track argmax
18	DONE_STATE	Signal completion

3.3.1 Convolution Implementation

Each convolutional output requires multiple clock cycles:

Layer 1 (per output position):

- 1 cycle: Load bias
- 1 cycle: Wait for BRAM
- 1 cycle: Prefetch
- 9 cycles: 3×3 MAC operations
- 1 cycle: Save result

- Total: 13 cycles per position $\times$ 10,816 positions = 140,608 cycles

Layer 2 (per output position):

- 144 cycles: $3 \times 3 \times 16$ MAC operations
- Total: ~ 150 cycles per position $\times$ 3,872 positions = 580,800 cycles

3.3.2 Max Pooling Implementation

The max pooling operation uses a 5-step pipeline per 2×2 window:

1. Set address for value 0
2. Read value 0, set address for value 1
3. Compare with max, set address for value 2
4. Compare with max, set address for value 3
5. Compare with max, write result

3.4 Memory Architecture

3.4.1 Distributed RAM

All working RAMs use **distributed memory** (LUT-based) for single-cycle combinational reads:

Listing 3.1: Distributed RAM Template

```

1 (* ram_style = "distributed" *)
2 reg [7:0] ram [0:SIZE-1];
3
4 // Combinational read (0-cycle latency)
5 assign rd_data = ram[rd_addr];
6
7 // Synchronous write
8 always @(posedge clk) begin
9     if (wr_en) ram[wr_addr] <= wr_data;
10 end

```

This eliminates pipeline bubbles that would occur with Block RAM's registered outputs.

3.4.2 Memory Organization

Table 3.3: Memory Allocation

Memory	Size (bytes)	Purpose
Conv Weights RAM	4,752	L1+L2 convolutional weights
Conv Biases RAM	192	48×32 -bit biases
Dense Weights RAM	8,000	FC layer weights
Dense Biases RAM	40	10×32 -bit biases
Buffer A	10,816	L1 conv output ($16 \times 26 \times 26$)
Buffer B	2,704	Pool outputs
Image RAM	784	Input image
Scores RAM	40	Output scores
Total	~ 27 KB	

3.5 Communication Protocol

3.5.1 UART Router

The system employs a dedicated **UART Router** module to manage signal multiplexing between the UART receiver and multiple consumer modules (weight loader, image loader, command handlers). The router’s primary functions are:

- **Packet Detection:** Recognize protocol marker sequences (two-byte start/end markers)
- **Signal Routing:** Route decoded payloads to appropriate destination modules (weight_loader or image_loader) based on detected marker type
- **Flow Control:** Forward data bytes only when destination modules are ready, maintaining protocol synchronization

The router uses payload-aware marker detection to ensure robustness: end markers are validated only after the complete payload (12,984 bytes for weights, 784 bytes for image) has been received. This prevents false transitions triggered by marker-like byte sequences in binary data.

Operation is gated by a `weights_loaded` status signal that enforces protocol ordering: weight loading is allowed only when `weights_loaded = 0`, while image loading and command processing are allowed only when `weights_loaded = 1`. This prevents out-of-sequence operations and ensures the inference engine has valid weights before attempting inference.

Command bytes (0xCC for predicted digit, 0xCD for all class scores) are processed immediately in the idle state and forwarded to dedicated command readers for result transmission via UART TX.

3.5.2 UART Configuration

- Baud Rate: 115,200 bps
- Frame Format: 8N1 (8 data bits, no parity, 1 stop bit)
- Clock Frequency: 100 MHz

3.5.3 Protocol Markers

Table 3.4: Protocol Markers

Marker	Bytes	Direction	Purpose
0xAA 0x55	Start	PC → FPGA	Begin weight transfer
0x55 0xAA	End	PC → FPGA	End weight transfer
0xBB 0x66	Start	PC → FPGA	Begin image transfer
0x66 0xBB	End	PC → FPGA	End image transfer
0xCC	Command	PC → FPGA	Request predicted digit
0xCD	Command	PC → FPGA	Request all scores

3.5.4 Binary Safety

The protocol uses payload-length-aware parsing to prevent false end-marker detection in binary data:

Listing 3.2: Binary-Safe Marker Detection

```
1 // Only check end markers AFTER receiving expected payload
2 if (byte_count >= WEIGHT_SIZE &&
3     prev_byte == WEIGHT_END1 &&
4     rx_data == WEIGHT_END2) begin
5     state <= DONE_W;
6 end
```

3.5.5 Flow Control

Python scripts use chunked transmission to prevent UART buffer overflow:

Listing 3.3: Chunked Transmission

```
1 def send_chunked(ser, data, chunk_size=32, delay=0.010):
2     for i in range(0, len(data), chunk_size):
3         chunk = data[i:i+chunk_size]
4         ser.write(chunk)
5         ser.flush()
6         time.sleep(delay) # 10ms delay per chunk
```

Chapter 4

Verification

This chapter describes the comprehensive verification methodology ensuring bit-exact correspondence between Python simulation and FPGA hardware.

4.1 Verification Strategy

The verification approach consists of three levels:

1. **Python Bit-Exact Simulation:** A Python implementation that exactly replicates FPGA arithmetic
2. **Verilog Testbench:** RTL simulation with test vectors
3. **Hardware Testing:** FPGA testing via UART with comparison framework

4.2 Python Bit-Exact Simulation

The Python simulation replicates every step of the FPGA arithmetic pipeline:

Listing 4.1: Bit-Exact Convolution Simulation

```
1 def convolve_layer(input_vol, weights, biases, shift):
2     for each output position:
3         acc = bias + sum(input * weight)
4
5         # FPGA-exact pipeline:
6         acc = acc >> shift          # Arithmetic right shift
7         if acc < 0: acc = 0          # ReLU
8         if acc > 127: acc = 127      # Saturation
9
10    output = acc
```

Key aspects of bit-exact simulation:

- Use of Python integers (arbitrary precision) with explicit clipping
- Arithmetic right shift ($\gg$) for signed division
- Explicit saturation to 8-bit range
- Same memory layout and addressing as FPGA

4.3 Test Infrastructure

4.3.1 Comparison Framework

Script automates testing:

1. Load and preprocess MNIST image
2. Run Python simulation to get expected scores
3. Send image to FPGA via UART
4. Wait for inference completion ($\sim 150\text{ms}$)
5. Request scores via `0xCD` command
6. Compare bit-exact match and accuracy

4.4 Verilog Testbench

Isolates and tests the `inference` module independently, bypassing the system-level UART stack. Test vectors are generated offline from a Python reference implementation that performs bit-exact arithmetic replication of the FPGA's quantized pipeline. For each test image, the Python model computes all 10 class logits and these results are stored in hexadecimal memory files alongside input pixel data and quantized weights.

The testbench loads 100 consecutive images from the MNIST dataset. For each image, it loads the pixel data and weights from memory files, executes the FPGA inference module, and compares the computed class scores against the Python-generated golden reference scores. The console output displays the per-image logit comparison, the FPGA-predicted digit, and the ground-truth label, enabling quick verification that the FPGA inference matches the Python reference across the test set.

4.5 Debug Infrastructure

4.5.1 Status LEDs

The design includes 16 debug LEDs with latching behavior:

Table 4.1: Debug LED Assignments

LED	Function
0	Weights loaded
1	Image loaded
2	Start pulse triggered (latched)
3	Done pulse triggered (latched)
4	Non-zero pixel read (latched)
5	Non-zero weight read (latched)
6	Non-zero bias read (latched)
7	Buffer A write enable (latched)
8	Non-zero L1 output (latched)
9	Buffer B write enable (latched)
10	Scores write enable (latched)
15	Heartbeat (blink)

4.5.2 7-Segment Display

The display shows:

- Left digit: Last stored prediction from RAM
- Right digit: Current predicted digit output

Chapter 5

Results and Comparison

This chapter presents the experimental results and comparison with simpler models and related work.

5.1 Classification Accuracy

5.1.1 Primary Results

Testing on the MNIST test set yields:

Table 5.1: Classification Accuracy Results

Implementation	Accuracy	Test Size
PyTorch (FP32)	99.2%	10,000 images
Python Simulation (INT8)	96.6%	3,000 images
FPGA Hardware	96.6%	3,000 images

The 2.6% accuracy drop from quantization is acceptable for practical embedded applications.

5.1.2 Bit-Exact Verification

Comparison between Python simulation and FPGA hardware shows:

- **Prediction Match:** 100% agreement on all test images
- **Score Match:** Bit-exact correspondence for all 10 class scores
- **Verification:** 3,000 images tested with zero discrepancies

5.2 Resource Utilization

Table 5.2: FPGA Resource Utilization (Artix-7 XC7A35T)

Resource	Used	Available	Utilization
LUTs (Logic)	~2,000	20,800	~10%
LUTs (RAM)	~27,000	33,280	~81%
Flip-Flops	~1,500	41,600	~4%
Block RAM	0	50 (36Kb)	0%
DSP Slices	0	90	0%

Notable observations:

- Distributed RAM dominates resource usage
- No DSP slices used (pure LUT-based multiplication)
- No Block RAM used (all distributed for low latency)

5.3 Performance Analysis

5.3.1 Timing

Table 5.3: Inference Timing Breakdown

Stage	Operations	Est. Cycles
Conv Layer 1	97,344 MACs	~140,000
Pool Layer 1	2,704 windows	~14,000
Conv Layer 2	557,568 MACs	~580,000
Pool Layer 2	800 windows	~4,000
Dense Layer	8,000 MACs	~8,000
Total	662,912 MACs	~750,000 cycles

At 100 MHz: ~7.5 ms per inference, or ~133 images/second throughput.

5.3.2 Power Consumption

Estimated power consumption based on similar Artix-7 implementations:

- Static Power: ~0.1 W
- Dynamic Power: ~0.3 W
- Total: ~0.4 W

5.4 Comparison with Simpler Models

Table 5.4: Comparison with Simpler Architectures

Model	Parameters	Accuracy	Memory
Softmax Regression	7,850	~92%	~8 KB
2-Layer MLP (784-128-10)	101,770	~97%	~100 KB
This Work (2-Layer CNN)	12,810	96.6%	~27 KB

Key observations:

- CNN achieves highest accuracy with moderate parameter count
- Convolutional weight sharing reduces memory vs. equivalent MLP
- Spatial feature extraction provides significant accuracy boost

5.5 Comparison with Related Work

Table 5.5: Comparison with Published FPGA Implementations

Work	Platform	Precision	Accuracy	Power
This Work	Artix-7	INT8	96.6%	~0.4 W
[3]	Artix-7	1-bit	84.0%	0.617 W
[4]	Virtex-5	–	97.2%	–
[5]	Zynq-7000	12-bit	97.6%	–
[6]	–	–	94.3%	5.0 W

This implementation achieves competitive accuracy among comparable low-power FPGA implementations while maintaining moderate resource requirements and significantly lower power consumption.

5.6 Summary

The implemented CNN achieves:

- **96.6% accuracy** on MNIST classification
- **Bit-exact** correspondence between Python and FPGA
- **~7.5 ms** inference latency (133 images/second)
- **~27 KB** total memory footprint
- **~0.4 W** estimated power consumption

These results demonstrate the viability of deploying quantized CNNs on resource-constrained FPGAs for embedded inference applications.

Bibliography

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [2] "A survey of FPGA-based accelerators for convolutional neural networks," *Neural Computing and Applications*, 2018.
- [3] "Binary Neural Network Implementation for Handwritten Digit Recognition on FPGA," *arXiv:2512.19304*, 2024.
- [4] "Deep neural network accelerator," *IEEE Xplore*, 2017.
- [5] "A Hardware Accelerator for The Inference of a Convolutional Neural Network," *Redalyc*, 2021.
- [6] "Hardware implementation of FPGA-based spiking attention neural network accelerator," *PMC*, 2024.
- [7] "Post-training quantization for efficient FPGA-based neural network acceleration," *ScienceDirect*, 2025.
- [8] "Basys3 FPGA Board Reference Manual," Digilent Inc.
- [9] "Vivado Design Suite User Guide: Synthesis," Xilinx UG901.